

Executive Production Readiness Report

ExamForge AI | Generated: 2026-08-02

Overall Production Score

The following scores represent the current state of the ExamForge AI Next.js application based on comprehensive audits across code quality, security, performance, accessibility, data integrity, realtime, and offline capabilities. Each score is calculated from the number and severity of findings in each category, with Critical findings weighted most heavily. Scores are out of 100.

Category	Score	Status
Security	42/100	NOT PRODUCTION READY - 2 Critical, 4 High findings
Performance	48/100	NOT PRODUCTION READY - 3 Critical, 8 High findings
Accessibility	52/100	NOT PRODUCTION READY - 4 Critical, 12 High findings
Data Integrity	38/100	NOT PRODUCTION READY - 8 Critical, 8 High findings
Maintainability	55/100	NEEDS WORK - 2 Critical, 6 High findings
Scalability	35/100	NOT PRODUCTION READY - Unbounded queries, N+1 patterns
UI Consistency	65/100	NEEDS WORK - Inconsistent states, missing loading states
Production Readiness	42/100	NOT PRODUCTION READY

Production Readiness Score: 42/100

The ExamForge AI application is NOT production-ready in its current state. There are 19 Critical findings across all audit categories that must be resolved before enterprise deployment. The most impactful areas requiring immediate attention are: (1) Data leakage bugs where school admins and teachers can see cross-school data, (2) Broken role-based access control in middleware, (3) Missing Content Security Policy, and (4) No authorization checks on server actions. The application has a solid architectural foundation with proper Supabase integration, but the implementation quality gaps are significant enough to prevent safe production deployment.

Critical Blockers (Must Fix Before Deployment)

#	Blocker	Category	Impact
1	Data leakage: school admins see cross-school data	Data	Data security breach
2	Data leakage: teachers see all pending grading	Data	Data security breach
3	Reports not filtered by school	Data	Data security breach
4	Teacher dashboard counts exams instead of students	Data	Incorrect business metrics
5	Analytics date range filter never applied	Data	Incorrect analytics
6	Middleware RBAC route prefixes do not match	Security	Role-based access control broken
7	No Content Security Policy	Security	XSS, clickjacking vulnerability
8	Server actions lack authorization checks	Security	Unauthorized data modification
9	N+1 queries in reports (250+ queries)	Performance	Timeouts at scale

#	Blocker	Category	Impact
10	Unbounded queries (no pagination)	Scalability	Memory issues, slow load times
11	App layout is entirely client component	Performance	No RSC benefits, slow FCP
12	Duplicate realtime subscriptions	Realtime	Double processing, wasted resources
13	No onAuthStateChange listener	Realtime	Session desync after token refresh
14	No offline indicator or mutation queue	Offline	Silent failures when offline
15	Sortable table headers not keyboard-accessible	Accessibility	WCAG 2.1.1 violation
16	No skip-link in authenticated layout	Accessibility	WCAG 2.4.1 violation
17	Color-only status indicators	Accessibility	WCAG 1.4.1 violation
18	No prefers-reduced-motion support	Accessibility	WCAG 2.3.3 violation
19	Auth callback open redirect	Security	Phishing/redirect vulnerability

Summary by Audit Category

Code Audit: 18 findings. 14+ unused npm packages, 5x duplicate utility functions, dead API client module, unused Prisma client, 34 production console statements. The middleware RBAC is broken because route prefixes do not match actual routes.

Security Audit: 12 findings. No CSP, server actions lack authorization, auth callback open redirect, no rate limiting, no security headers. Positive: getUser() used correctly in middleware, SSR client properly configured, service role key never exposed.

Performance Audit: 27 findings. Entire app layout is a client component, analytics and notifications pages use client-side data fetching, recharts not dynamically imported (~200KB), TanStack Query configured but never used, missing per-route loading.tsx files.

Accessibility Audit: 36 findings. Sortable table headers not keyboard-accessible, no skip-link in authenticated layout, color-only status indicators, no prefers-reduced-motion support, search inputs without labels, icon-only buttons missing accessible names.

Data Integrity Audit: 26 findings. 8 Critical data leakage/correctness bugs, 4 N+1 query patterns that will cause timeouts, 6 unbounded queries without pagination, analytics date range filter never applied, silent error handling in search service.

Realtime/Offline Audit: 17 findings. Duplicate realtime subscriptions, no onAuthStateChange, no reconnection data reconciliation, useOffline hook never used, Dexie never configured, no offline mutation queue or network recovery handler.

Recommended Phase 6.1 Priority Actions

Based on the audit findings, the following actions should be taken in priority order before the application can be considered production-ready. These are grouped into three phases: P0 (must fix before any deployment), P1 (must fix before enterprise deployment), and P2 (should fix for production quality).

P0: Must Fix Before Any Deployment

1. Fix data leakage bugs (scope queries by school/teacher) | 2. Fix middleware RBAC route prefixes | 3. Add Content Security Policy | 4. Add authorization checks to server actions | 5. Fix

analytics date range filter | 6. Fix teacher dashboard student count | 7. Validate auth callback next parameter | 8. Add pagination to all list queries

P1: Must Fix Before Enterprise Deployment

9. Fix N+1 queries in reports service | 10. Split app layout into Server + Client | 11. Refactor analytics/notifications to Server Components | 12. Dynamic import recharts | 13. Remove unused packages | 14. Add onAuthStateChanged listener | 15. Eliminate duplicate realtime subscriptions | 16. Add rate limiting to API routes | 17. Replace console.error/warn with structured logging | 18. Add per-route loading.tsx | 19. Add keyboard accessibility to data tables | 20. Add skip-link to authenticated layout

P2: Should Fix for Production Quality

21. Consolidate duplicate utility functions | 22. Delete dead code (api/, db.ts, toast-service, etc.) | 23. Use TanStack Query for client-side fetching | 24. Add prefers-reduced-motion support | 25. Add aria-labels to all interactive elements | 26. Wire up useOffline and add offline banner | 27. Implement Dexie-based offline mutation queue | 28. Add reconnection data reconciliation | 29. Add conflict resolution for exam answers | 30. Add security headers to API responses